



Assignment – 2

Design and Analysis of Algorithms (IT2005)
Indian Institute of Information Technology Allahabad

Instructor: K P Singh/Navjot/Naveen

Instructions

- All answers must be clearly justified. Hand written solution is desirable.
- Algorithms must include time and space complexity analysis.
- Use pseudo-code where appropriate.
- Late submissions will not be consider for evaluation.
- Plagiarism will be checked. AI generated submission will be penalized.

Question 1. We want to evaluate a degree $n - 1$ polynomial $P(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1}$ at a given point $x = \alpha$.

Input: integers $a_0, a_1, \dots, a_{n-1}$ in an array. And an integer α .

Output: $a_0 + a_1\alpha + a_2\alpha^2 + \dots + a_{n-1}\alpha^{n-1}$

Design an algorithm for this which uses only $O(n)$ multiplications and additions.

Question 2. You are given a function $f : \{1, 2, \dots, n\} \rightarrow \{1, 2, \dots, n\}$ (it is given as an integer array of size n where all the entries are between 1 and n . i th entry in the array is $f(i)$). You need to find the largest subset $S \subseteq \{1, 2, \dots, n\}$ such that f is one-one and onto when restricted to S . In other words, (1) every element in S is mapped to an element in S and (2) no two elements in S are mapped to the same element in S . Design an $O(n)$ algorithm for this.

Example input: 4, 6, 2, 3, 5, 4.

Output: 2, 3, 4, 5, 6

Question 3. Given a set of intervals we want to count the number of intervals which are not contained in any other interval. For example, interval (2, 4) is contained in (2, 5). Interval (2, 4) is contained in (1, 6). Interval (1, 4) is not contained in (2, 5). Interval (1, 4) is not contained in (0, 3).

You can assume input is two integer arrays L and R of size n each. $L[i]$ is the left endpoint and $R[i]$ is the right endpoint of the i th interval. Design an $O(n \log n)$ algorithm for this.

Example input: $L = [5, 3, 8, 11, 9, 7, 2, 15]$. $R = [10, 8, 12, 16, 20, 15, 13, 17]$.

Output: 3

Question 4. Given n integers and a number k , we want to compute the sum of products of all k size subsets. That is, given $a_1, a_2, \dots, a_n$, we want to compute

$$\sum_{i_1 < i_2 < \dots < i_k} a_{i_1} a_{i_2} \dots a_{i_k}.$$



Question 5. Non-dominated points. For points in 2 dimensions, we say (a, b) dominates (c, d) if $a \geq c$ and $b \geq d$. Given a set of n points in 2 dimensions, design an $O(n \log n)$ time algorithm to compute the set of points which are not dominated by any other point.

Question 6. Majority Fingerprints. You are given a collection of n fingerprints. You are also told that more than $(n/2)$ of these are identical to each other. You are only given access to an equality test, which takes two fingerprints and tells whether they are identical or not. Using this equality test, can you find out the one with more than $n/2$ copies in $O(n \log n)$ time?

Question 7. Significant inversions. In an array A of integers, a pair (i, j) is called a significant inversion if $i < j$ and $A[i] > 2A[j]$. Design an $O(n \log n)$ time algorithm to find the number of significant inversions.

Question 8. Hidden lines. Consider n lines in the 2D plane:

$$y = m_i x + c_i$$

for $1 \leq i \leq n$. We want to compute the set of lines which will be visible when one sees from $y = +\infty$.

Let us elaborate. At an x -coordinate x_0 , that line will be visible which has maximum the y coordinate among all lines. That is, the line which maximizes $m_i x_0 + c_i$. A line is called hidden if it is not visible at any x -coordinate.

Design an $O(n \log n)$ time algorithm to find the set of visible (which are not hidden) lines. Assume the input is given as $(m_1, c_1), (m_2, c_2), \dots$.

Question 9. Design a divide and conquer based algorithm ($O(n \log n)$ time) for the significant inversion problem.

Question 10. Augmented binary search tree means the usual BST with each node storing some additional information. This additional information can be useful in designing algorithms for certain problems. Which of the following information can be updated appropriately in $O(\log n)$ time, when a node is inserted or deleted in the BST? Assume that the height of the tree is $O(\log n)$.

- each node x stores the number of nodes in the subtree rooted at x .
- each node x stores the number of nodes larger than x .
- each node x stores its distance from the root.
- each node x stores a pointer to its successor (that is the smallest number larger than x).
- each node x stores the largest depth of a node in the subtree rooted at x .
- each node x stores its rank in the BST.

Question 11. Can we compute the n th Fibonacci number in $O(\log n)$ additions and multiplications?

Question 12. Consider an n -node complete binary tree T , where $n = 2^d - 1$ for some integer d . Each node v of T is labelled with a real number x_v . You may assume that the real numbers labelling the nodes are all distinct. A node v in T is a *local minimum* if the label x_v is less than the label x_w for all nodes w that are joined to v by an edge. You are given such a complete binary tree T , but the labeling is specified only in the following implicit way: for each node v ,



you can determine the value x_v by probing the node v . Show how to find a local minimum of T using only $O(n)$ probes to the nodes of T .

Question 13. We are interested in analysing some hard-to-obtain data from two databases. Each database contains n numerical values (so there are $2n$ values in total). Assume that all these values are distinct. We would like to determine the *median* of these $2n$ values, which we define as the n th smallest value. However, the only way to access the values is through queries to the databases. In a single query, we specify a value k to one of the two databases, and the chosen database returns the k th smallest value that it contains. Give an algorithm that finds the median value using only $O(n)$ queries.

Question 14. Optional. Not Compulsory

This problem illustrates how to compute the Fourier Transform (FT) using modular arithmetic, for example modulo 7. There exists a number ω such that its powers

$$\omega^1, \omega^2, \omega^3, \omega^4, \omega^5, \omega^6$$

are all distinct modulo 7.

- (a) Find such a number ω , and show that

$$1 + \omega + \omega^2 + \omega^3 + \omega^4 + \omega^5 \equiv 0 \pmod{7}.$$

(Interestingly, for any prime modulus, such a number exists.)

- (b) Using the matrix form of the Fourier Transform, compute the transform of the sequence

$$(0, 1, 1, 1, 5, 2)$$

modulo 7. That is, multiply this vector by the matrix $M_6(\omega)$, where ω is the value found in part (a). Recall that the (j, k) entry of this matrix is

$$\omega^{jk}, \quad \text{where } 1 \leq j, k \leq 6.$$

All arithmetic should be performed modulo 7.

- (c) Write down the matrix required to compute the inverse Fourier Transform. Show that multiplying the transformed vector by this matrix returns the original sequence. (Again, all arithmetic should be performed modulo 7.)
- (d) Show how to multiply the polynomials

$$x^2 + x + 1 \quad \text{and} \quad x^3 + 2x - 11$$

using the Fourier Transform modulo 7.